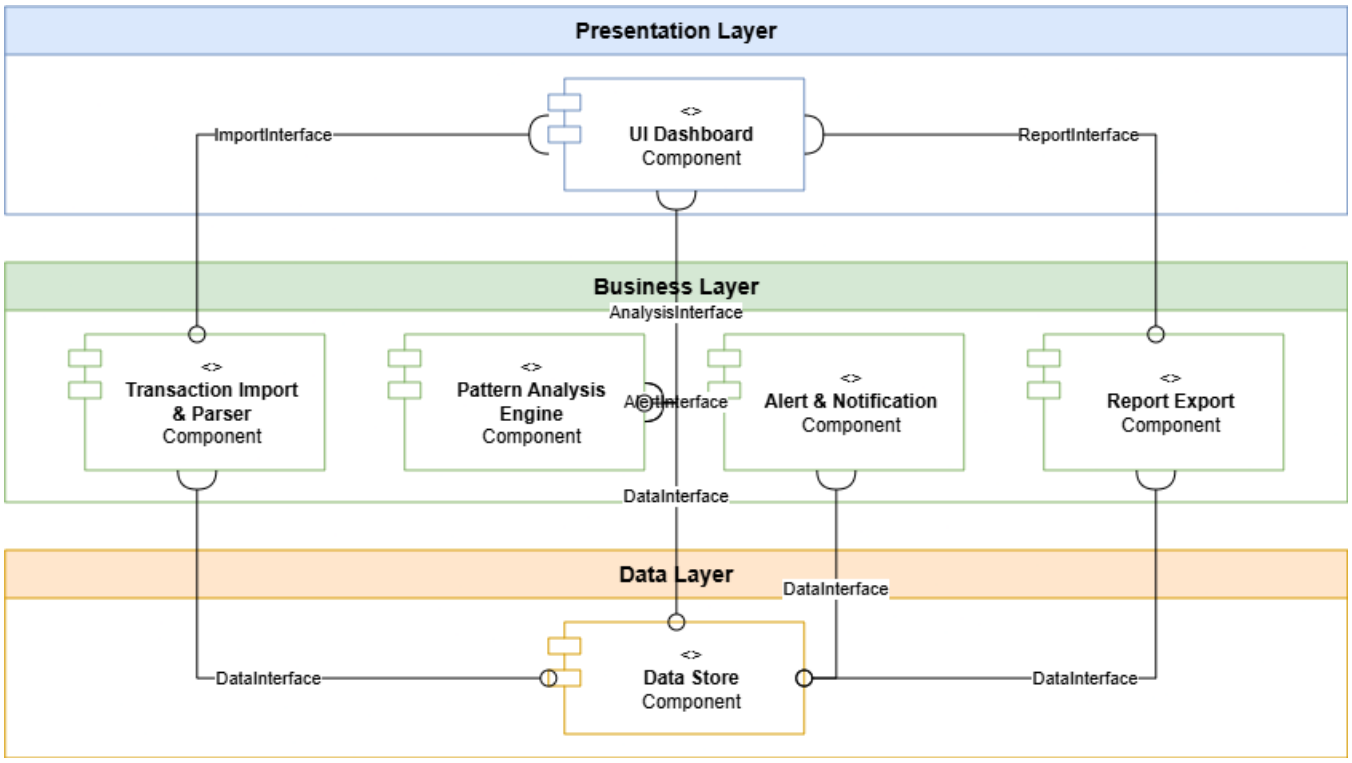
Lab 3: Architectural Pattern Selection — Justification

Student: Abdul Mateen Shaikh | SRN: PES1UG24AM332 | Section: F



Architecture Selection: *"We chose Layered Architecture for the Personal Subscription & Recurring Expense Auditor System."*

Reason 1 — Clear separation between import, analysis, and alerting logic

The system performs a sequence of distinct, dependent responsibilities: importing and parsing bank transaction files, detecting recurring billing patterns and calculating burn rate, and sending renewal alerts or cancellation guidance. A Layered Architecture (Presentation, Business, Data) cleanly separates the touch-screen/web dashboard from this business logic and from data storage, which matches the way requirements were already decomposed into use cases (UC1–UC8) and keeps each concern independently testable and modifiable.

Reason 2 — Predictable, single-user workload

The Subscription & Expense Auditor is a personal finance tool used by one individual (or a small finance-auditor role) at a time, analyzing a bounded transaction history (60 days minimum, up to 5,000 transactions). This workload does not need the independent scaling or fault isolation that Microservices provide, nor does it need the strict server-centric control of a Client-Server model; a single, well-layered application is simpler to build, deploy, and maintain for this scale.

Security Advantage

Layering enforces that the Presentation (UI Dashboard) component never touches the Data Layer directly — it must go through the Business Layer's Transaction Import, Pattern Analysis, or Report Export components. This lets sensitive operations, such as encrypting imported transaction data (UC2) before it is stored, be centralized and validated in one place rather than being reachable from multiple entry points.

Performance Benefit

Because the Data Layer is isolated behind a single Data Store component with a well-defined interface, the Pattern Analysis Engine can cache or index transaction records for repeated recurring-pattern checks without other layers needing to know how storage is implemented — directly supporting the target of analyzing a 12-month, 5,000-transaction history in under 5 seconds.